VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



ASSIGNMENT 1 REPORT

FOUNDATIONS OF DEEP LEARNING PIPELINES AND ARCHITECTURES

From Linear Models to Modern Sequence Models: A Comparative Study for Image Classification

Course: CO3133 – Deep Learning and Its Applications
Semester: 261
Instructor: Lê Thành Sách
Class: CC01
Group: Choco

Name	Student ID
Nguyễn Minh Quang	2453052
Nguyễn Minh Phúc	2453005
Nguyễn Thanh Thế	2453191

Ho Chi Minh City, October 2026

Contents

1 Problem and Data Description **4**
1.1 Problem Formulation 4
1.2 Dataset Identity and Scope 4
1.3 Exploratory Data Analysis 5
1.4 Data Splitting and Leakage Controls 6
1.5 Preprocessing 6
1.6 Dataset and DataLoader 6

2 Methodology **6**
2.1 Shared Experimental Pipeline 6
2.2 Common Training and Evaluation Protocol 7
2.3 Linear Classifier 8
2.4 Multilayer Perceptron 9
2.5 Convolutional Neural Network 10
2.6 Gated Recurrent Unit 11
2.7 Transformer Encoder 12
2.8 Experimental Design 13

3 Implementation Results **14**
3.1 Training Behavior 14
3.2 Quantitative Results 16
3.3 Qualitative Results 17
3.4 Comparison and Discussion 19
3.5 Error Analysis 19
3.6 Ablation Study 20
3.7 Limitations 21
3.8 Conclusion and Future Work 22

4 Reproducibility and Project Artifacts **22**

5 Contributions and AI Usage Disclosure **23**
5.1 Member Contributions 23
5.2 AI Usage Disclosure 23

List of Figures

1 Class distribution from the saved Fashion-MNIST EDA. 5
2 Representative Fashion-MNIST images from the EDA. 5
3 Shared training and evaluation pipeline. 7
4 Architecture of the CNN. Each block shows the tensor shape (channels × height × width) after the stage; stacked sheets indicate several feature maps. 10
5 Image rows (schematic at left) and a one-layer GRU unrolled over 28 timesteps. The same GRU parameters are reused at every step, and only h_{28} feeds the classifier. 11
6 Row-token Transformer architecture. The expanded encoder layer is repeated twice before mean pooling. Skip arrows bypass self-attention and the feed-forward network, respectively. 12
7 Mean training and validation dynamics across the three seeds. Solid lines denote validation, dashed lines denote training, and shaded regions show ±1 sample standard deviation. Each model is shown through its last epoch shared by all three seeds. 15
8 CNN confusion matrix for the seed-42 test evaluation. 17
9 Examples correctly classified by the CNN on the seed-42 test set; parentheses show predicted-class confidence. 18
10 Examples misclassified by the CNN on the seed-42 test set, including visually similar upper-body garments and footwear. 18

List of Tables

1 Dataset split used in all experiments. 6
2 Common experimental configuration. 8
3 MLP forward shapes and trainable parameters per layer. 9
4 Model comparisons, hypotheses, and assessment criteria. 14
5 Saved benchmark run inventory. 15
6 Test-set predictive performance across three training seeds (mean ± sample standard deviation). 16
7 Model size and computational cost; inference time is mean ± sample standard deviation across three seeds. 16
8 Seed-42 class-wise recall percentages across the five models. 19
9 Six most frequent CNN confusion pairs on the seed-42 test set. 20
10 Seed-42 representation ablation using best validation accuracy. 21
11 Member contributions to Assignment 1. 23
12 AI assistance used during Assignment 1. 24

1 Problem and Data Description

1.1 Problem Formulation

Neural networks can solve the same classification problem using different assumptions about how input information is represented. A linear classifier treats pixels as a flattened vector. An MLP adds nonlinear feature composition but also starts from a flattened image. A CNN preserves the two-dimensional layout and reuses local filters, whereas recurrent and attention-based models can process image rows as ordered sequences. Comparing these families on one controlled dataset helps examine how representation choices relate to predictive performance and computational cost.

Fashion-MNIST classification assigns a clothing category to each grayscale product image. A system that can distinguish visually similar garments is a useful small-scale setting for studying how different neural architectures represent images. Each input is one 28×28 image x; the target $y \in \{0, \ldots, 9\}$ is one of ten classes. The prediction unit is the image, not an individual pixel or image region. For a model with parameters θ, the predicted class is

$$\hat{y}(x) = \arg \max_{k \in \{0,\ldots,9\}} f_\theta(x)_k, \tag{1}$$

where $f_\theta(x) \in \mathbb{R}^{10}$ contains unnormalized class scores (logits). We optimize multiclass cross-entropy and compare predicted labels with the ground truth using accuracy and macro-F1.

The main question is which architecture performs best under a shared data split, training setup, and evaluation protocol, and what trade-offs emerge between predictive performance, model size, and computational cost. Linear and MLP models flatten pixels, CNNs preserve local spatial structure, and the GRU and Transformer process image rows as ordered tokens. Classes such as Shirt, T-shirt/top, Pullover, and Coat can have similar low-resolution silhouettes, making class-level errors important alongside aggregate metrics. The comparison uses saved runs across three training seeds to examine both performance and variation between runs.

1.2 Dataset Identity and Scope

The main dataset is Fashion-MNIST [1], introduced by Xiao, Rasul, and Vollgraf and released by Zalando Research. Its official dataset repository is available at `https://github.com/zalandoresearch/fashion-mnist`. It contains 70,000 labeled images: 60,000 in the official training partition and 10,000 in the official test partition. Each image has one grayscale channel and dimensions 28×28 pixels. The ten labels are T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot. The original distribution uses IDX image and label files, including gzip-compressed copies. Our local compressed files occupy approximately 30.9 MB in total. The dataset was loaded through `torchvision.datasets.FashionMNIST`; the project records torchvision 0.20.1 as the package version rather than assigning a separate release number to the dataset. The upstream Fashion-MNIST repository publishes the dataset under the MIT license.

These are product images at a fixed, low resolution. They support a controlled comparison of model families, but results on this dataset alone cannot establish performance on color photographs, different camera conditions, or real-world retail image distributions. The published dataset provides one class label per image; it does not provide object boxes or garment attributes for a finer error audit.

1.3 Exploratory Data Analysis

The recorded EDA confirms a common image shape of $[1, 28, 28]$ and ten classes. The official 60,000-image training partition is balanced; after the saved stratified split, the training subset contains 5,000 images per class and validation contains 1,000 per class. The official test set also contains 1,000 per class. Thus, class-frequency imbalance is not a confounder in the main comparison. This count does not establish equal visual diversity within each class.

Raw image pixels are unsigned 8-bit values; `ToTensor` scales them to floating-point values in $[0, 1]$. On the 50,000 training images after this scaling, the recorded mean is 0.286276 and population standard deviation is 0.353185. These training-subset statistics are used by the main runs for mean–standard-deviation normalization. Figure 1 presents the class counts and figure 2 gives a visual overview of the ten categories. The examples suggest that upper-body garment classes deserve particular attention in the later confusion-matrix analysis; the actual frequency of those confusions is assessed in section 3.

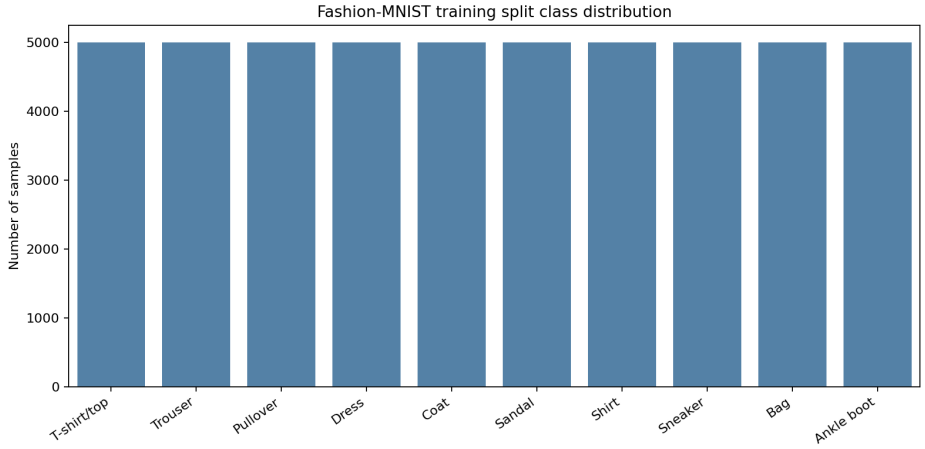


Figure 1: Class distribution from the saved Fashion-MNIST EDA.

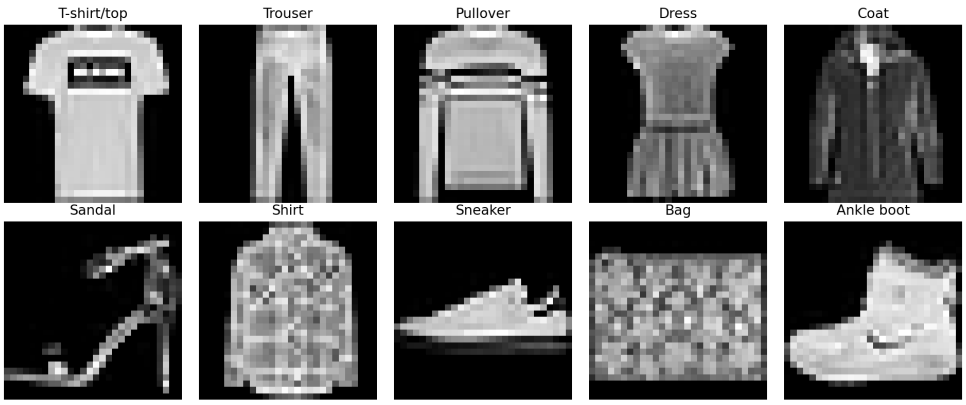


Figure 2: Representative Fashion-MNIST images from the EDA.

1.4 Data Splitting and Leakage Controls

The official test set remains separate. We split the official 60,000 training images into 50,000 training and 10,000 validation images with scikit-learn's stratified `train_test_split`, using split seed 42. The selected indices are saved in a manifest and reused across all five model families and all training seeds. The loader validates the manifest's split seed, expected sizes, index bounds, complete coverage of the original training partition, and absence of overlap between train and validation indices. The sample, rather than a person or acquisition session, is the available split unit.

Validation data determine the best checkpoint and early stopping. The official test set is used for final evaluation after checkpoint selection. Training seeds 42, 123, and 2026 alter initialization and batch ordering; they do not recreate the data partition. No subset of the saved training or validation indices was selected for the main benchmark.

Table 1: Dataset split used in all experiments.

Split	Samples	Per class	Purpose
Training	50,000	5,000	Parameter fitting
Validation	10,000	1,000	Checkpoint selection
Official test	10,000	1,000	Final evaluation

1.5 Preprocessing

The main protocol applies `ToTensor` and no resize: Fashion-MNIST images already have the required 28×28 dimensions. The resulting values are standardized using the mean 0.286276 and population standard deviation 0.353185 computed from the saved 50,000-image training subset. The same fixed statistics are applied to training, validation, and test images. Random augmentation is disabled. This preprocessing is shared across all architectures so that the model comparison changes the architecture while keeping the input representation fixed. Fashion-MNIST supplies a valid class index for each image in the loaded dataset; no invalid-annotation removal is part of this pipeline.

1.6 Dataset and DataLoader

The dataset implementation creates separate training, validation, and test `DataLoader` objects. Training batches are shuffled using a generator seeded by the run seed; validation and test batches keep a fixed order. The recorded runs use batch size 128, `num_workers=0`, and pinned memory. For a full batch after preprocessing, the images have shape $[128, 1, 28, 28]$ and type `float32`; the 128 class targets are integer labels of type `int64` and shape [128]. The final batch of a split may be smaller because the loader does not drop it.

2 Methodology

2.1 Shared Experimental Pipeline

All five models use the same dataset module, train/validation loop, loss function, checkpoint logic, and evaluation code. The data module loads the saved split and produces image–label

batches. During training, the selected architecture maps images to ten logits, cross-entropy measures the disagreement with labels, and Adam updates parameters by backpropagation. After each epoch, the validation loop records loss, accuracy, and macro-F1 without updating weights. The best validation checkpoint is saved for final test inference. The test evaluator converts logits to labels by arg max, computes the same classification metrics, saves predictions and a confusion matrix, and measures forward-pass time. There is no task-specific post-processing beyond the class decision. The project implements the model compositions, split-manifest checks, training loop, checkpoint policy, and evaluation workflow. It uses PyTorch modules such as `nn.Linear`, `nn.Conv2d`, `nn.GRU`, `nn.TransformerEncoderLayer`, and `DataLoader`; scikit-learn supplies the stratified splitting routine and classification metrics.

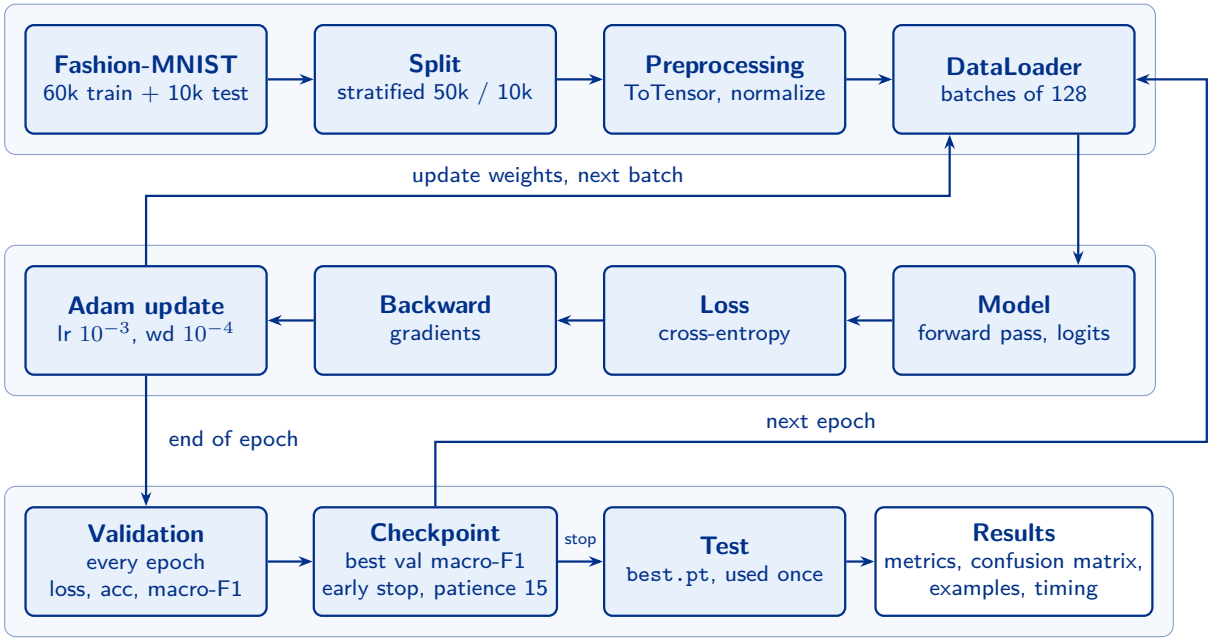


Figure 3: Shared training and evaluation pipeline.

2.2 Common Training and Evaluation Protocol

Every model outputs raw logits and is trained with PyTorch's `CrossEntropyLoss`. For an image with target class y and logits $z_0, \ldots, z_9$, the per-image loss is

$$\ell(z, y) = -z_y + \log\left(\sum_{k=0}^{9} e^{z_k}\right). \tag{2}$$

No model applies softmax before this loss. Epoch losses are weighted by the number of samples in each batch. Accuracy is the fraction of correct predictions. Macro-F1 is the arithmetic mean of the ten per-class F1 values, calculated from all predictions in a split with zero division set to zero. This metric gives each class equal weight in the reported score, even when error rates differ by class.

The fixed training protocol uses Adam with learning rate 10^{-3} and weight decay 10^{-4}, batch size 128, and at most 100 epochs. There is no learning-rate scheduler. After each epoch, validation macro-F1 is compared with the best recorded value. A change larger than `min_delta`=0.0001 is an improvement; values within that range are resolved using lower validation loss. An exact remaining tie keeps the earlier checkpoint. Training stops after 15 successive epochs without

a qualifying improvement, or at the 100-epoch cap. The final test evaluation loads `best.pt`, rather than the weights from the last epoch. The training seeds are 42, 123, and 2026; the saved data-split seed remains 42 for every run.

Table 2: Common experimental configuration.

Setting	Value
Loss	Cross-entropy on logits
Optimizer	Adam
Learning rate	0.001
Weight decay	0.0001
Batch size	128
Maximum epochs	100
Early stopping	Validation macro-F1; patience 15; $\Delta = 0.0001$
Training seeds	42, 123, 2026
Hardware	NVIDIA GeForce RTX 4050 Laptop GPU
Precision	Float32

The project records Python 3.11.15, PyTorch 2.5.1+cu121 and torchvision 0.20.1+cu121 for the benchmark environment; dependency versions are also listed in the repository. Python, NumPy, and PyTorch generators are seeded for each run, and deterministic algorithms are requested. This improves repeatability on the same software and device, but does not guarantee identical results on other library versions or hardware.

Training time is measured by a wall-clock timer around the epoch loop, including training, validation, and checkpoint work in that loop. Inference time is a separate forward-only measurement on the test loader: five batches warm up the model, then three full-loader passes are timed. Inputs are transferred to the GPU before each timed forward call, and CUDA is synchronized on both sides of that call. The recorded milliseconds per sample are the mean timed forward duration divided by the number of evaluated images; this is not single-image latency or end-to-end application time.

These hyperparameters were fixed as a shared comparison protocol. The project has not established them as individually optimal for each architecture through a systematic tuning search. Architectural choices, such as dropout and representation size, remain model-specific and are documented below.

2.3 Linear Classifier

The Linear classifier serves as the simplest reference model: it flattens each grayscale image $x \in \mathbb{R}^{1\times28\times28}$ into a vector $v \in \mathbb{R}^{784}$ and directly computes ten class scores (logits) through one affine transformation:

$$z = Wv + b, \qquad W \in \mathbb{R}^{10\times784}, \quad b \in \mathbb{R}^{10}, \quad z \in \mathbb{R}^{10}. \tag{3}$$

Each logit is a weighted sum of the 784 pixel values plus a class-specific bias. For a mini-batch, the tensor shapes are $[B, 1, 28, 28] \to [B, 784] \to [B, 10]$, where B is the number of images. The predicted class is the index of the largest logit. These raw logits are passed directly to the shared cross-entropy loss; no separate softmax layer is applied before the loss calculation.

The `Linear`$(784, 10)$ layer learns one weight for every input–output pair and one bias for each output class. Its trainable parameter count is therefore

$$P_{\text{Linear}} = 784 \times 10 + 10 = \boxed{7{,}850}. \tag{4}$$

With no hidden layer or nonlinear activation, the model forms affine decision boundaries between classes. Its small parameter count makes it an inexpensive baseline, but flattening does not explicitly encode pixel neighborhoods or reuse learned detectors across image locations. Consequently, it cannot directly model general nonlinear interactions between garment features.

2.4 Multilayer Perceptron

The Multilayer Perceptron (MLP) starts from the same flattened image $v \in \mathbb{R}^{784}$, but inserts two fully connected hidden layers before the classification layer. The chosen architecture is $784 \to 256 \to 128 \to 10$. The hidden widths 256 and 128 are architectural hyperparameters rather than dimensions required by the dataset; 784 comes from the input image and 10 from the number of classes. Each hidden affine transformation is followed by ReLU, $\text{ReLU}(a) = \max(0, a)$, and dropout with probability 0.2 during training.

For one image, the forward computation is

$$h_1 = \text{ReLU}(W_1 v + b_1), \qquad \tilde{h}_1 = \text{Dropout}_{0.2}(h_1), \tag{5}$$

$$h_2 = \text{ReLU}(W_2 \tilde{h}_1 + b_2), \qquad \tilde{h}_2 = \text{Dropout}_{0.2}(h_2), \tag{6}$$

$$z = W_3 \tilde{h}_2 + b_3, \qquad z \in \mathbb{R}^{10}. \tag{7}$$

Here $W_1 \in \mathbb{R}^{256\times784}$, $W_2 \in \mathbb{R}^{128\times256}$, and $W_3 \in \mathbb{R}^{10\times128}$; the bias vector of each layer has one element per output neuron. Thus, each hidden neuron forms a new weighted combination of *all* activations from the preceding layer: the transition $784 \to 256$ is learned, not a selection or deletion of 528 pixels. Dropout randomly zeros some hidden activations during training and is disabled during validation and test inference.

Table 3: MLP forward shapes and trainable parameters per layer.

Stage	Operation	Output shape	Parameters
Input	Flatten	$[B, 784]$	0
Hidden 1	Linear $784 \to 256$, ReLU, dropout 0.2	$[B, 256]$	200,960
Hidden 2	Linear $256 \to 128$, ReLU, dropout 0.2	$[B, 128]$	32,896
Classifier	Linear $128 \to 10$	$[B, 10]$	1,290
		Total	**235,146**

For a fully connected layer, the number of trainable parameters is $(n_{\text{in}} + 1)n_{\text{out}}$: the first term counts weights, and the extra 1 represents each output neuron's bias. Therefore,

$$P_{\text{MLP}} = (784 + 1) \times 256 + (256 + 1) \times 128 + (128 + 1) \times 10 = \boxed{235{,}146}. \tag{8}$$

ReLU and dropout contribute no trainable parameters. ReLU is essential for expressiveness: without nonlinear activations, successive affine layers can be collapsed into a single affine mapping, leaving the network functionally equivalent to a Linear classifier. The MLP can therefore learn more complex pixel interactions, although its larger capacity raises computational and overfitting concerns. Like the Linear baseline, it still lacks explicit spatial locality and weight sharing across image positions.

2.5 Convolutional Neural Network

Unlike the flattened-input baselines, the CNN retains the image as a single-channel spatial tensor $x \in \mathbb{R}^{1\times 28\times 28}$. A convolution learns local filters that are reused across positions. For output channel j, the feature map can be expressed as

$$Y_j = \sum_{c=1}^{C_{\text{in}}} X_c \star K_{j,c} + b_j, \tag{9}$$

where $\star$ denotes the cross-correlation implemented by `Conv2d`, $K_{j,c}$ is a learned spatial kernel, and b_j is the channel bias. With kernel size k, padding p, stride s, and unit dilation, the output height is

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1, \tag{10}$$

with the analogous expression for width. For both convolutions, $k = 3$, $p = 1$, and $s = 1$, so the spatial dimensions are preserved. Each subsequent 2×2 max-pooling operation uses stride 2 and reduces $28 \to 14$ or $14 \to 7$.

The architecture comprises two `Conv2d-ReLU-MaxPool2d` blocks, followed by a fully connected classification head. Omitting the batch dimension, its feature shapes are

$$\begin{aligned} 1 \times 28 \times 28 &\xrightarrow{\text{Conv}(1,32)} 32 \times 28 \times 28 \xrightarrow{\text{Pool}} 32 \times 14 \times 14 \\ &\xrightarrow{\text{Conv}(32,64)} 64 \times 14 \times 14 \xrightarrow{\text{Pool}} 64 \times 7 \times 7 \\ &\xrightarrow{\text{Flatten}} 3136 \xrightarrow{\text{Linear+ReLU}} 128 \xrightarrow{\text{Dropout}_{0.3}} 128 \xrightarrow{\text{Linear}} 10. \end{aligned} \tag{11}$$

Figure 4 displays the same sequence; channel counts correspond to numbers of learned feature maps rather than independent copies of the original image. The output is ten logits.

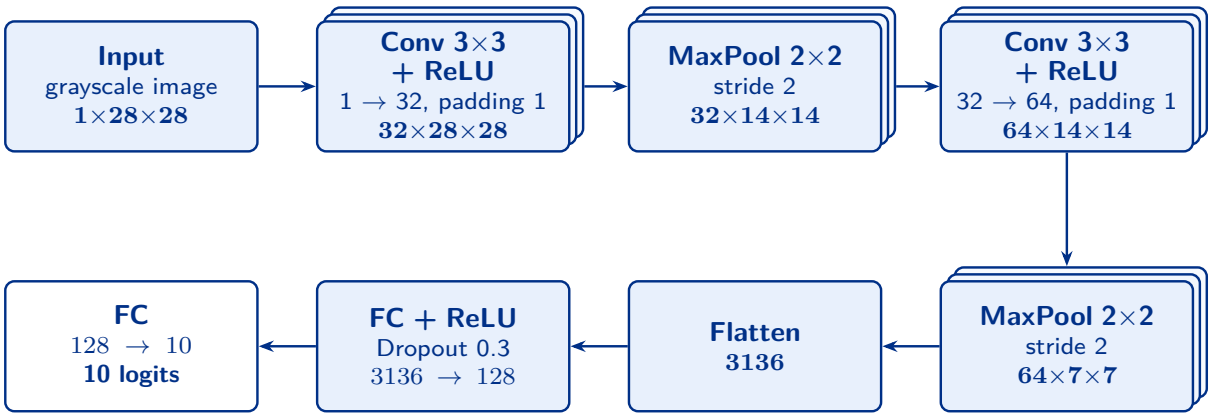


Figure 4: Architecture of the CNN. Each block shows the tensor shape (channels × height × width) after the stage; stacked sheets indicate several feature maps.

Both convolutional layers include biases. Their parameter counts, together with the two fully connected layers, are

$$\begin{aligned} P_{\text{CNN}} &= (3 \cdot 3 \cdot 1 + 1) \cdot 32 + (3 \cdot 3 \cdot 32 + 1) \cdot 64 \\ &\quad + (3136 + 1) \cdot 128 + (128 + 1) \cdot 10 \\ &= 320 + 18{,}496 + 401{,}536 + 1{,}290 = \boxed{421{,}642}. \end{aligned} \tag{12}$$

The first fully connected layer alone accounts for approximately 95.2% of this total, while both convolutions together account for about 4.5%. Thus, although this is the largest of the five tested models by parameter count, most parameters belong to its classifier rather than its spatial filters. Local connectivity and weight sharing encourage recognition of garment edges and shapes across image positions. However, pooling can discard fine details, and the large flattened classification head increases model size.

2.6 Gated Recurrent Unit

The recurrent model interprets a 28×28 image as a sequence of 28 horizontal rows, processed from top to bottom. Removing the singleton channel dimension gives a batch tensor $[B, 28, 28]$; at timestep $t \in \{1, \ldots, 28\}$, the input $x_t \in \mathbb{R}^{28}$ is row t. One unidirectional GRU layer has hidden size 128. Its initial state h_0 is a zero vector and each subsequent state $h_t \in \mathbb{R}^{128}$ summarizes information retained from the rows observed so far.

A GRU updates its hidden state through reset and update gates. Using a standard conceptual formulation, these operations are

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r), \qquad u_t = \sigma(W_u x_t + U_u h_{t-1} + b_u), \tag{13}$$

$$\widetilde{h}_t = \tanh\big(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h\big), \tag{14}$$

$$h_t = u_t \odot h_{t-1} + (1 - u_t) \odot \widetilde{h}_t, \tag{15}$$

where σ is the sigmoid, $\odot$ denotes elementwise multiplication, r_t controls the use of the previous hidden state when constructing a candidate, and u_t determines the relative contribution of the old state. PyTorch's `nn.GRU` uses a closely related candidate-state implementation in which the reset gate is applied after the hidden-state affine transform; the schematic equations above explain the gate roles rather than claiming literal identity with that library's internal formula.

For the implemented model, the forward mapping and classifier are

$$h_t = \mathrm{GRU}(x_t, h_{t-1}), \quad z = W h_{28} + b, \qquad W \in \mathbb{R}^{10 \times 128}. \tag{16}$$

Across a batch, the sequence of hidden outputs has shape $[B, 28, 128]$; selecting its final timestep gives $[B, 128]$, which the linear head maps to $[B, 10]$. Figure 5 shows the row representation and the GRU unrolled over time. Each illustrated GRU cell uses the *same* learned weights; there are 28 processing steps, not 28 independently parameterized GRU layers.

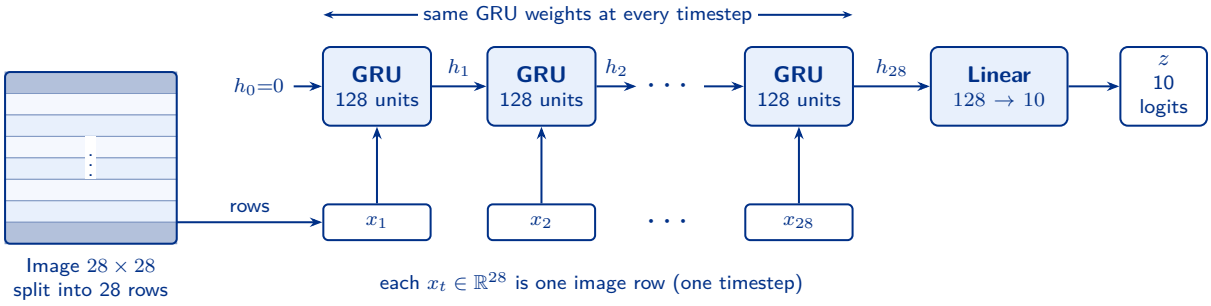


Figure 5: Image rows (schematic at left) and a one-layer GRU unrolled over 28 timesteps. The same GRU parameters are reused at every step, and only h_{28} feeds the classifier.

PyTorch allocates separate input-to-hidden and hidden-to-hidden bias vectors for each of the

three gates. The trainable-parameter count, including the ten-class linear head, is

$$\begin{aligned} P_{\text{GRU}} &= 3(128 \cdot 28 + 128 \cdot 128 + 2 \cdot 128) + (128 + 1) \cdot 10 \\ &= 60{,}672 + 1{,}290 = \boxed{61{,}962}. \end{aligned} \tag{17}$$

The hidden state supports accumulation of information across image rows without a large dense input layer. Nevertheless, reading a static 2D image as a 1D sequence imposes an artificial processing order; later rows depend on preceding recurrent steps, which also limits parallel execution across timesteps. The model uses no recurrent dropout.

2.7 Transformer Encoder

The Transformer uses the same 28-row image representation as the GRU, but processes relationships among rows with self-attention rather than a recurrent hidden state. An image is represented as $X \in \mathbb{R}^{28\times28}$, containing 28 row tokens of 28 pixel values each. A shared learned linear projection maps each token to 64 features, followed by a learned positional embedding:

$$H_0 = \text{Linear}_{28\to64}(X) + P, \qquad H_0 \in \mathbb{R}^{28\times64}, \quad P \in \mathbb{R}^{28\times64}. \tag{18}$$

The linear projection includes its bias. In the batched model, P has broadcast shape $[1, 28, 64]$; it identifies a token's row position, not a physical relocation of pixel values. No patch splitting or special classification (CLS) token is used.

The embedded tokens pass through two Transformer encoder layers. Each layer contains four-head self-attention, residual connections, layer normalization, and a token-wise feed-forward network $64 \to 128 \to 64$ with GELU and dropout 0.1. The encoder preserves shape $[B, 28, 64]$. Figure 6 shows the row-token input, the expanded encoder layer repeated twice, and the mean-pooling classification head. In the expanded block, each skip connection bypasses its own sublayer before the corresponding Add & Norm operation.

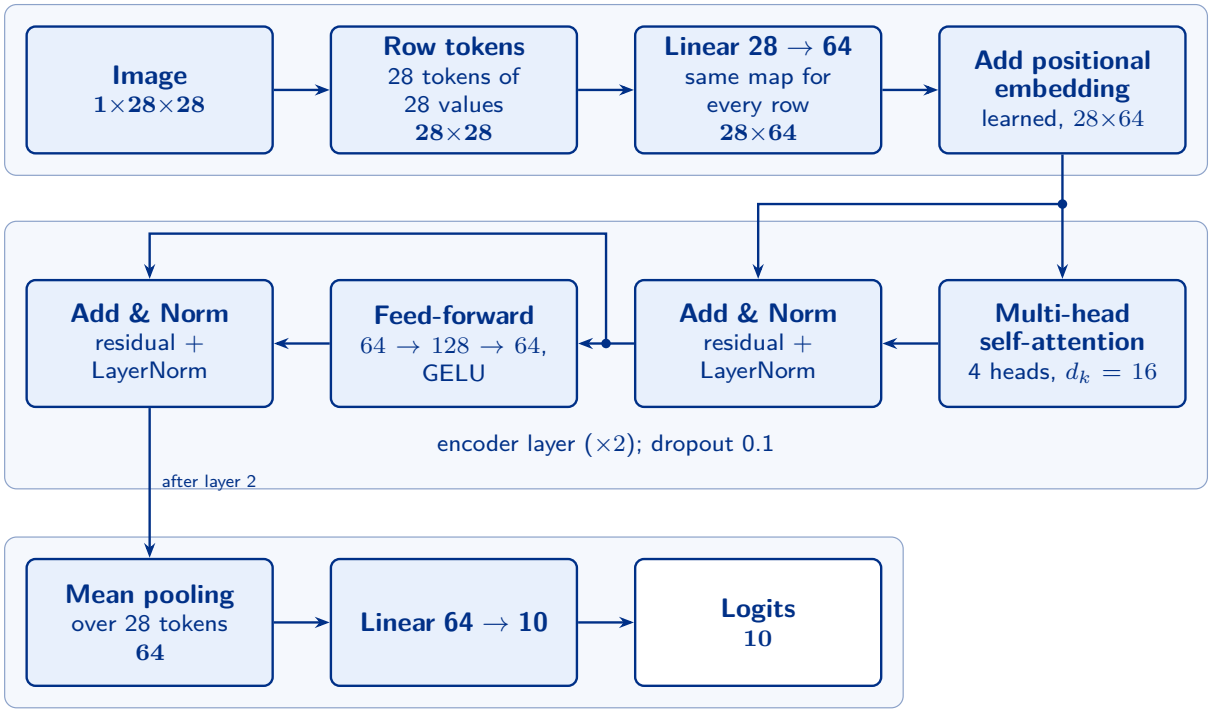


Figure 6: Row-token Transformer architecture. The expanded encoder layer is repeated twice before mean pooling. Skip arrows bypass self-attention and the feed-forward network, respectively.

For one attention head, the learned query, key, and value projections are formed from its input tokens. Scaled dot-product self-attention is

$$A = \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}}\right), \qquad \text{Attention}(Q, K, V) = AV. \tag{19}$$

With embedding dimension 64 and four heads, $d_k = 64/4 = 16$; for a single image and head, $Q, K, V \in \mathbb{R}^{28\times 16}$ and $A \in \mathbb{R}^{28\times 28}$. Thus, every row token compares its query with all 28 keys. The row-wise softmax normalizes the attention weights across keys, and the weighted values update each token representation. The four heads are concatenated and projected back to 64 dimensions. The feed-forward network applies the same nonlinear transform to each token; self-attention is the operation that exchanges information between different tokens.

After the second encoder layer, mean pooling produces a single image representation, and a linear classifier returns ten logits:

$$h_{\text{img}} = \frac{1}{28}\sum_{t=1}^{28} H_{2,t}, \qquad z = W h_{\text{img}} + b, \quad W \in \mathbb{R}^{10\times 64}. \tag{20}$$

In batch notation, the final steps are $[B, 28, 64] \to [B, 64] \to [B, 10]$. The trainable-parameter breakdown is

$$\begin{aligned} P_{\text{Transformer}} &= \underbrace{(28+1)\cdot 64}_{\text{input projection}} + \underbrace{28\cdot 64}_{\text{position embeddings}} \\ &\quad + \underbrace{2\cdot 33{,}472}_{\text{encoder layers}} + \underbrace{(64+1)\cdot 10}_{\text{classifier}} \\ &= 1{,}856 + 1{,}792 + 66{,}944 + 650 = \boxed{71{,}242}. \end{aligned} \tag{21}$$

Attention provides direct interaction between distant image rows and the compact row-token representation keeps the model relatively small. However, each token summarizes an entire horizontal slice; this architecture does not encode local two-dimensional regions as explicitly as CNN filters or a patch-based Transformer. Its small size and row tokenization are design choices, not evidence that this configuration is optimal for Fashion-MNIST.

2.8 Experimental Design

The main experiment compares five model families: Linear, MLP, CNN, GRU, and Transformer. The primary changed factor is the model architecture. All runs use the same saved Fashion-MNIST training, validation, and test splits, training-only normalization statistics, cross-entropy loss, optimizer settings, batch size, early-stopping and checkpoint-selection rules, and evaluation procedure described in section 2.2. Each architecture is evaluated using the same three training seeds (42, 123, and 2026). This shared protocol reduces variation caused by the data and training setup.

The comparisons in table 4 examine whether different ways of representing and processing the same images are associated with differences in predictive performance and computational cost. The assessment focuses on the implemented configurations rather than on theoretical capabilities of each model family.

Table 4: Model comparisons, hypotheses, and assessment criteria.

Comparison	Hypothesis	Assessment criteria
Linear vs. MLP	Nonlinear hidden layers can model pixel interactions beyond a single affine mapping.	Compare test macro-F1 and accuracy; consider the additional parameter and runtime costs.
MLP vs. CNN	Local convolutions and shared filters can benefit recognition of garment shapes compared with dense features from flattened pixels.	Compare predictive scores, parameter counts, and recurring class-level errors.
MLP vs. GRU	Reading image rows sequentially may capture relationships that a flattened MLP does not explicitly model.	Compare macro-F1, accuracy, parameter counts, and inference time.
GRU vs. Transformer	Recurrent state updates and self-attention offer different ways to integrate information across the same 28 image rows.	Compare predictive scores and computational costs without attributing differences to recurrence or attention alone.
CNN vs. Transformer	Global interactions between row tokens do not necessarily outperform convolutional spatial features on small images.	Compare macro-F1 and accuracy alongside model size, timing, and error patterns.

Macro-F1 is the primary predictive comparison metric, with accuracy as a complementary measure. For each model, we report the mean and sample standard deviation across the three seeds. Higher mean test macro-F1 indicates better observed predictive performance under this benchmark protocol; differences are interpreted alongside seed-to-seed variation and accuracy rather than treated as proof of statistical significance. Trainable-parameter count, total training time, and forward-only inference time are reported separately to describe accuracy–cost trade-offs. Validation macro-F1 determines the selected checkpoint within each run; test results are not used for checkpoint selection or hyperparameter tuning.

These comparisons are not single-factor ablations. Architecture-specific representations, widths, regularization, and parameter counts change together, so an observed difference cannot be assigned uniquely to one component. The conclusions are therefore limited to the five evaluated configurations and the recorded hardware and training conditions; controlled ablations would be needed to isolate individual design choices.

3 Implementation Results

3.1 Training Behavior

The final benchmark contains one completed run for each of the three training seeds 42, 123, and 2026 for every model family. Each run uses the same saved stratified split, normalized inputs, optimizer settings, validation rule, and checkpoint-selection protocol. The table below records the selected checkpoint range across the three seeds. The range is useful for describing training behavior, but it is not itself a quality ranking: an earlier selected checkpoint can reflect faster optimization, while a later one can reflect slower improvement or validation fluctuations.

Table 5: Saved benchmark run inventory.

Model	Training seeds	Best-epoch range
Linear	42, 123, 2026	9–35
MLP	42, 123, 2026	31–42
CNN	42, 123, 2026	16–21
GRU	42, 123, 2026	25–35
Transformer	42, 123, 2026	16–47

Training loss decreases quickly during the first epochs for all model families, followed by slower improvement and validation fluctuations. The selected checkpoints therefore occur before the 100-epoch limit. The CNN reaches its selected region relatively early and maintains the strongest validation trajectory, while the Linear model reaches a lower-capacity plateau. The row-based GRU and Transformer continue to depend on the sequence representation and attention or recurrence dynamics, so their validation curves can fluctuate even when training loss continues to fall. The mean line and the ±1 sample-standard-deviation band summarize the three seeds; they show stability of the general trend, not a replacement for per-seed values.

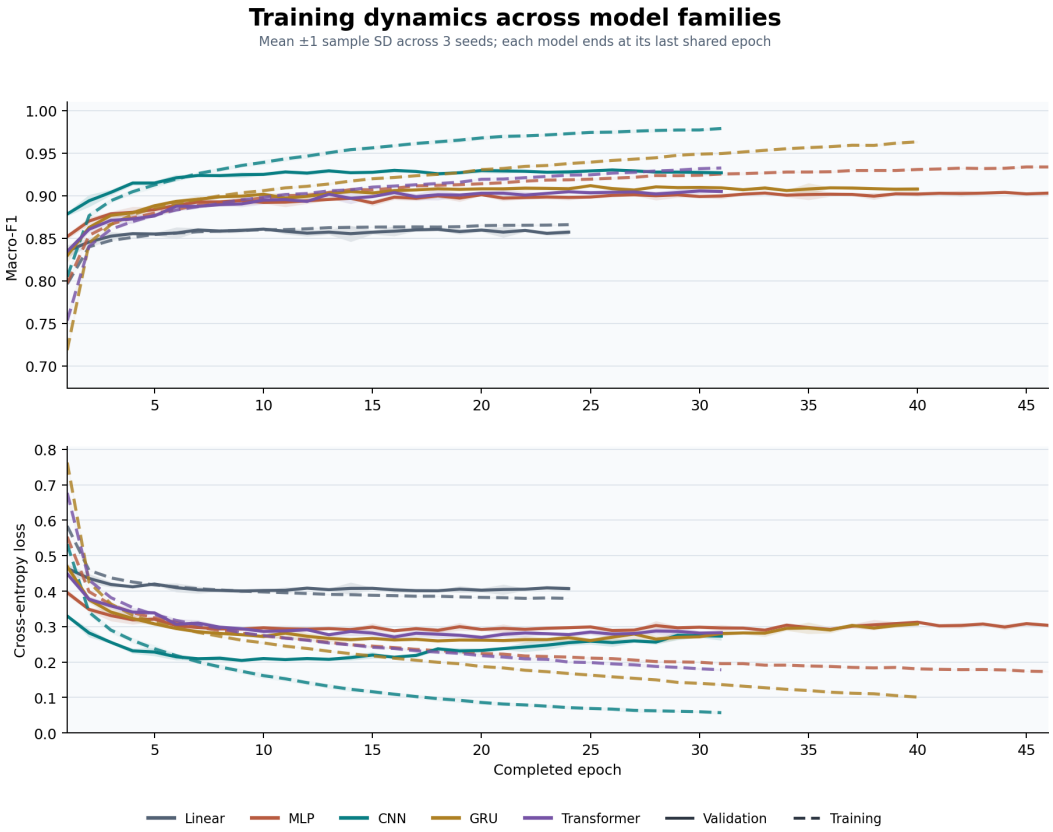


Figure 7: Mean training and validation dynamics across the three seeds. Solid lines denote validation, dashed lines denote training, and shaded regions show ±1 sample standard deviation. Each model is shown through its last epoch shared by all three seeds.

The gap between training and validation trajectories is interpreted as a possible generalization gap rather than proof of a specific failure cause. The trajectories support statements about convergence, plateauing, and seed-to-seed variation. They do not by themselves identify whether

a gap is caused by capacity, regularization, optimization noise, or the data. The validation-based checkpoint rule is therefore important: the final test prediction uses the best recorded validation state instead of automatically using the last epoch.

3.2 Quantitative Results

Table 6 compares predictive quality, while Table 7 separately compares model size and computational cost. Reported test accuracy and macro-F1 values are the mean and sample standard deviation across seeds 42, 123, and 2026.

Table 6: Test-set predictive performance across three training seeds (mean ± sample standard deviation).

Model	Accuracy	Macro-F1
Linear	$84.31 \pm 0.09\%$	$84.23 \pm 0.06\%$
MLP	$89.37 \pm 0.11\%$	$89.31 \pm 0.12\%$
CNN	$92.22 \pm 0.39\%$	$92.20 \pm 0.41\%$
GRU	$90.29 \pm 0.19\%$	$90.26 \pm 0.21\%$
Transformer	$89.40 \pm 0.09\%$	$89.37 \pm 0.13\%$

The CNN is the strongest model on both primary metrics: 92.22% accuracy and 92.20% macro-F1. The agreement between the two metrics matters because macro-F1 gives every class equal weight; the CNN is therefore not winning only because of the aggregate number of easy classes. Its mean macro-F1 is 7.97 percentage points above Linear, 2.89 points above MLP, 1.94 points above GRU, and 2.83 points above Transformer. The standard deviations are all below 0.41 percentage points for the primary metrics, so the ordering is consistent across the three seeds even though the three runs are not a formal test of statistical significance.

The MLP improves substantially over the Linear baseline, from 84.23% to 89.31% macro-F1, showing the benefit of nonlinear features. The CNN then adds a further gain while using spatially shared filters. The GRU reaches 90.26% macro-F1 with 61,962 parameters, making it the second strongest model in this comparison. The Transformer is close to the MLP at 89.37% macro-F1 and remains below the GRU. These results describe the implemented configurations; they do not prove that one architecture family is universally better under different widths, tokenizations, or tuning budgets.

The efficiency comparison is reported separately in Table 7. The parameter counts are fixed by each architecture, whereas measured inference times vary across the three runs.

Table 7: Model size and computational cost; inference time is mean ± sample standard deviation across three seeds.

Model	Parameters	Training time (s)	Inference (ms/sample)
Linear	7,850	—	0.00530 ± 0.00350
MLP	235,146	—	0.00781 ± 0.00833
CNN	421,642	—	0.01265 ± 0.01304
GRU	61,962	—	0.01880 ± 0.00734
Transformer	71,242	—	0.05044 ± 0.03408

Training-time values are not yet available for verified reporting; they are left blank rather than estimated.

Parameter count changes the interpretation of the accuracy ranking. Linear is the smallest

model with 7,850 parameters. The CNN has 421,642 parameters, more than the MLP, GRU, or Transformer, but its spatial inductive bias produces the best predictive result on this image dataset. The GRU is much smaller than the CNN while retaining competitive accuracy, which makes it a meaningful accuracy–capacity trade-off rather than a replacement for the best model on accuracy alone.

The inference numbers show a secondary cost pattern: Linear and MLP are the fastest, CNN is intermediate, GRU is slower than CNN in this implementation, and Transformer is slowest. The ranking is plausible given the different forward computations, but the standard deviations are large relative to some means, especially for MLP, CNN, and Transformer. That variation can come from GPU scheduling, memory state, short timing windows, and the interaction between batching and per-sample normalization. The measurement protocol is appropriate for a controlled implementation-level comparison, but inference time should not override the much more stable accuracy and macro-F1 evidence.

3.3 Qualitative Results

The CNN is used for qualitative analysis because it obtains the highest mean test accuracy and macro-F1. Figure 8 shows its confusion matrix on the seed-42 test evaluation. As a single-seed diagnostic, its entries should be distinguished from the three-seed means in Table 6. The diagonal shows correct classifications; off-diagonal entries reveal recurring confusions among clothing categories.

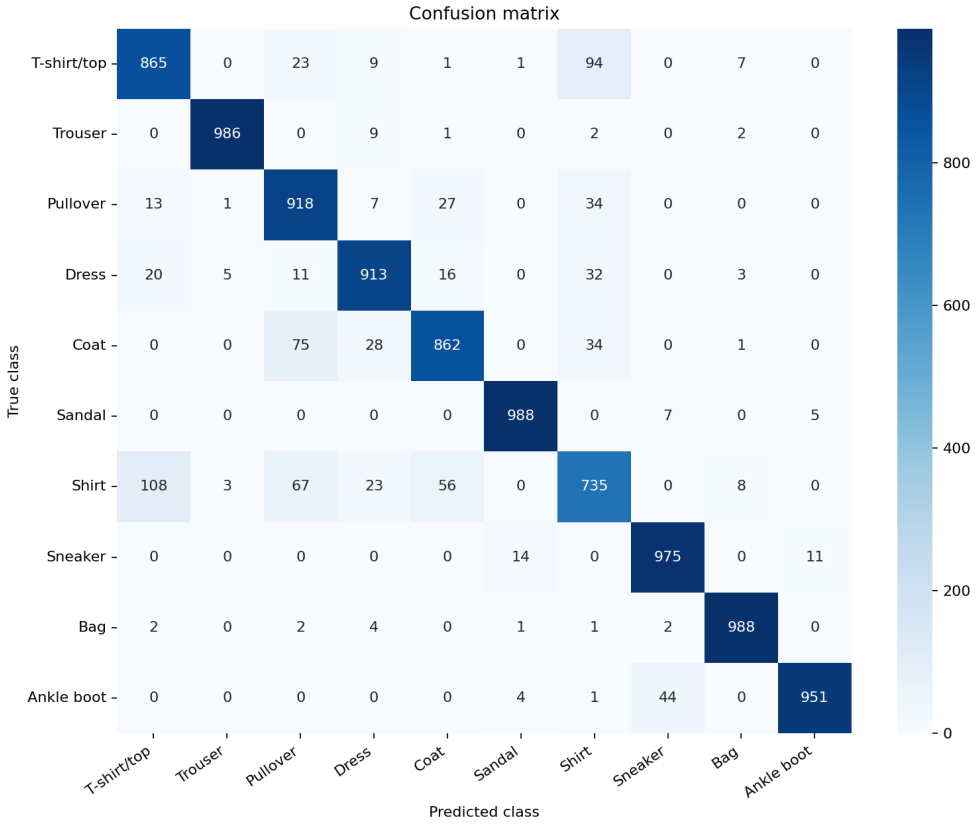


Figure 8: CNN confusion matrix for the seed-42 test evaluation.

Figures 9 and 10 show representative CNN predictions with true labels, predicted labels, and predicted-class confidence. Successful classifications include several Shirt and Coat examples despite visual similarity across upper-body garments. In contrast, the failures include Shirt classified as Coat or Pullover, Coat classified as Pullover, and Ankle boot classified as Sneaker or Sandal. These observations connect individual examples with the systematic mistakes visible in the matrix. High predicted-class confidence does not guarantee a correct label.

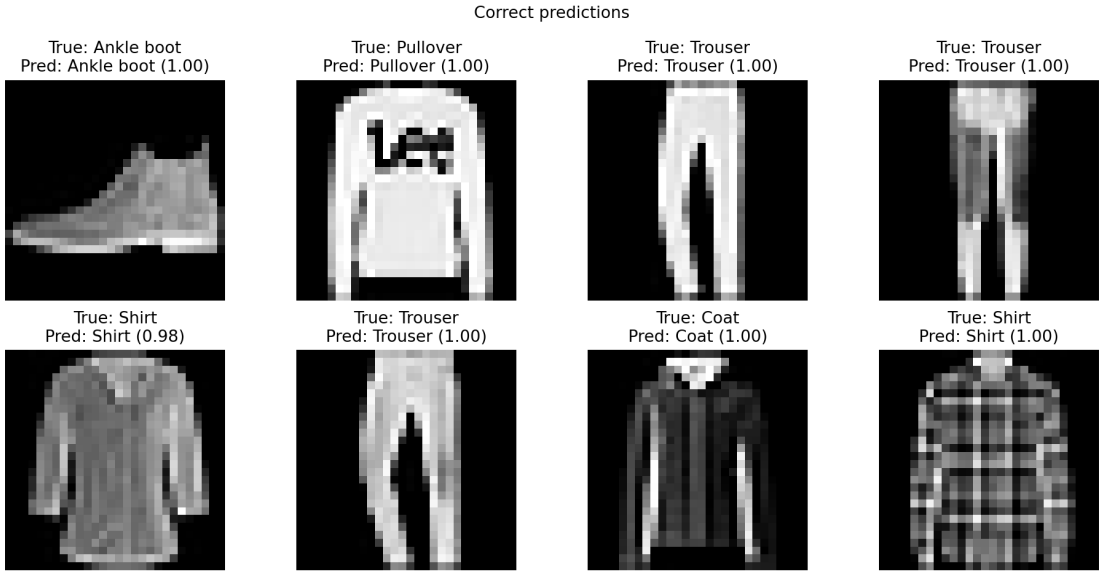


Figure 9: Examples correctly classified by the CNN on the seed-42 test set; parentheses show predicted-class confidence.

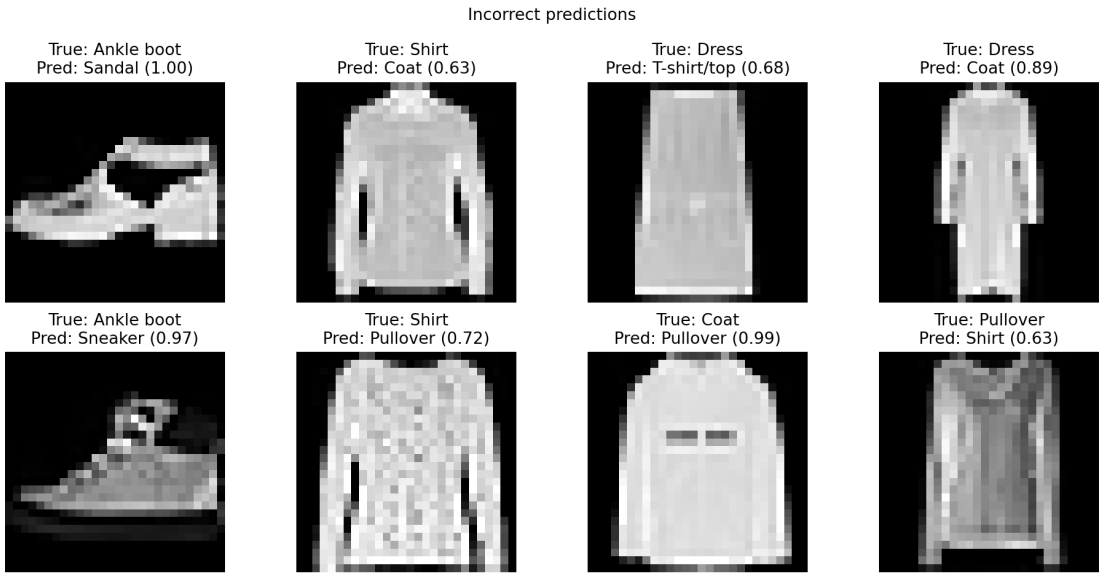


Figure 10: Examples misclassified by the CNN on the seed-42 test set, including visually similar upper-body garments and footwear.

3.4 Comparison and Discussion

The primary comparison favours the CNN. Its convolution and pooling operations preserve local spatial relationships and reuse filters across image locations, which matches the edges, contours, and local garment parts in Fashion-MNIST. The improvement from Linear to MLP shows that nonlinear feature combinations matter, while the further CNN gain suggests that flattening pixels does not provide the same spatial sharing. These claims are about the observed configurations, because parameter count and regularization also change between models.

The sequence models are informative but do not reverse the CNN result. The GRU processes 28 image rows through a recurrent hidden state and reaches 90.26% macro-F1. The Transformer connects row tokens through self-attention and reaches 89.37% macro-F1 with a similar parameter budget. The GRU is slightly stronger in this experiment, but the difference is small relative to the purpose of the comparison: it does not establish that recurrence is universally better than attention. It indicates that these row-token implementations were better or worse suited to this dataset and training protocol.

The model choice therefore depends on the criterion. CNN is the accuracy choice, GRU is a compact model with competitive predictive performance, and Linear or MLP offers lower measured inference cost in this implementation. Transformer does not provide a predictive advantage here despite its more expensive forward pass. No single scalar score is used to hide these trade-offs; accuracy and macro-F1 remain the primary criteria, while parameter count and inference time provide context.

3.5 Error Analysis

The first step is a class-wise comparison. Table 8 reports recall percentages for the seed-42 checkpoint of each model. Because the Fashion-MNIST test set has 1,000 examples per class, each value is the diagonal entry of that model's confusion matrix divided by 1,000. This table is a diagnostic view of class behavior; it is not a replacement for the three-seed aggregate metrics in Table 6.

Table 8: Seed-42 class-wise recall percentages across the five models.

Class	Linear	MLP	CNN	GRU	Transformer
T-shirt/top	79.7	83.4	86.5	87.6	83.6
Trouser	95.5	97.2	98.6	97.6	98.2
Pullover	74.7	82.6	91.8	85.6	84.2
Dress	84.7	92.0	91.3	88.3	90.3
Coat	74.9	83.8	86.2	85.1	88.3
Sandal	91.4	95.6	98.8	97.3	95.6
Shirt	59.1	70.3	73.5	70.4	63.8
Sneaker	94.0	94.9	97.5	97.1	96.7
Bag	93.8	97.0	98.8	97.5	97.2
Ankle boot	94.6	96.2	95.1	96.2	95.5

The CNN has the strongest recall on Pullover, Sandal, and Bag, while the Transformer is strongest on Trouser and Coat and the GRU is strongest on T-shirt/top. Shirt is the weakest class for every model, with recall ranging from 59.1% for Linear to 73.5% for CNN. This shows why the overall accuracy ranking should be read together with class-level behavior: a model can improve the aggregate score while still leaving one visually ambiguous class as its main weakness. For the CNN specifically, the seed-42 confusion matrix gives 86.5% recall for T-shirt/top, 98.6% for Trouser, 91.8% for Pullover, 91.3% for Dress, 86.2% for Coat, 98.8% for Sandal, 73.5% for Shirt, 97.5% for Sneaker, 98.8% for Bag, and 95.1% for Ankle boot.

Table 9 summarizes the six largest off-diagonal entries of the CNN's seed-42 confusion matrix. Each true class contributes 1,000 test images, so the per-class percentages are directly comparable.

Table 9: Six most frequent CNN confusion pairs on the seed-42 test set.

True class → Predicted class	Errors	Rate within true class
Shirt → T-shirt/top	108	10.8%
T-shirt/top → Shirt	94	9.4%
Coat → Pullover	75	7.5%
Shirt → Pullover	67	6.7%
Shirt → Coat	56	5.6%
Ankle boot → Sneaker	44	4.4%

Among the 819 CNN misclassifications in this seed, 532 (approximately 65.0%) are confusions within T-shirt/top, Pullover, Coat, and Shirt. This concentration shows that distinguishing upper-body garments is a more prominent difficulty than footwear recognition in the observed test set. The qualitative cases in Figure 10 illustrate some of these failures.

Upper-body errors occur in both directions: Shirt is often labeled as T-shirt/top, Pullover, or Coat, while examples from those classes are also predicted as Shirt. This pattern is consistent with similar silhouettes and weakly resolved collar or sleeve details in 28×28 grayscale images. These are plausible visual explanations, not direct measurements of the features the model has learned.

Footwear errors are less frequent but still structured. The matrix contains 44 Ankle boot images predicted as Sneaker and 14 Sneaker images predicted as Sandal. These mistakes are consistent with overlapping low-resolution silhouettes, while the high diagonal counts for Sandal and Sneaker show that the same representation is still effective for most footwear examples.

This analysis also explains why macro-F1 is reported alongside accuracy. A model can have a high overall accuracy while performing poorly on one class; Shirt recall shows that the CNN's 92.22% three-seed mean accuracy does not mean uniform performance across categories. Further improvements would need a controlled validation experiment, such as testing modest augmentation or preserving more spatial detail, rather than changing the training setup based only on the displayed matrix.

3.6 Ablation Study

This ablation isolates the image-to-sequence representation used by the two sequence models. The architecture, normalized preprocessing, data split, optimizer, learning rate, batch size, stopping rule, and seed are kept fixed; only the tokenization changes. The row representation treats each of the 28 image rows as one token, the column representation treats each of the 28

columns as one token, and the patch representation divides the image into 49 4×4 patches. This makes the comparison useful for checking whether the ordering and granularity of the input sequence affect performance, without changing the model family at the same time.

Because this is an initial ablation pass, the table reports the best validation accuracy from seed 42 rather than a three-seed estimate. The results are therefore used to identify a promising representation and to motivate further testing, not to claim a statistically stable ranking.

Table 10: Seed-42 representation ablation using best validation accuracy.

Model	Representation	Best validation accuracy
GRU	Rows	91.19%
GRU	Columns	90.31%
GRU	4×4 patches	89.67%
Transformer	Rows	90.63%
Transformer	Columns	90.29%
Transformer	4×4 patches	89.93%

The row representation is strongest for both model families in this seed-42 comparison. For the GRU, changing from rows to columns reduces accuracy by 0.88 percentage points, while using 4×4 patches reduces it by 1.52 points. This suggests that the recurrent model benefits from the 28-step sequence used by the row representation in this implementation. A row is a complete horizontal slice, so the recurrent state receives a simple and consistent scan of the image; changing to patches both changes the token content and increases the sequence length to 49 steps.

The Transformer shows the same ordering, although the differences are smaller. Columns are 0.34 percentage points below rows and patches are 0.70 points below rows. This indicates that self-attention does not remove the effect of tokenization: attention can compare tokens globally, but it still receives the spatial information through the chosen token boundaries and positional indices. The patch variant also changes the token dimension from 28 pixel values to 16, so its input projection and sequence geometry are different even though the rest of the training protocol is unchanged.

These results do not show that row sequences are universally better than columns or patches. They show that, for this Fashion-MNIST setup and this single seed, the existing row-based configuration is the safest choice among the tested representations. A stronger conclusion would require repeating all six configurations with the same three seeds and comparing the mean and standard deviation. Training time is deliberately excluded here because the purpose of this ablation is to examine predictive accuracy; runtime is also more sensitive to machine load and sequence length than the accuracy metric.

3.7 Limitations

The conclusions are specific to Fashion-MNIST, whose small, centered, 28×28 grayscale images do not represent the visual complexity of high-resolution, color, or unconstrained photographs. Although all five models share the data split, preprocessing, training protocol, and evaluation metrics, their representations, parameter counts, and architecture-specific regularization differ. Consequently, the observed ranking cannot be attributed to any single architectural component. The fixed configurations also do not establish the best achievable performance of each model family under broader hyperparameter tuning.

The three training seeds describe variation across the evaluated runs but are insufficient to establish universal or statistically significant superiority. Inference times were measured on one hardware environment and should not be generalized to other devices or deployment conditions. The representation ablation in section 3.6 used only seed 42 and best validation accuracy. Moreover, changing rows to 4×4 patches changes both the sequence length and input-token dimension. Its results therefore describe the tested configurations rather than isolating the effect of spatial tokenization alone. A learned classification token was not evaluated.

3.8 Conclusion and Future Work

Under the common Fashion-MNIST protocol, CNN achieves the highest three-seed mean test accuracy (92.22%) and macro-F1 (92.20%) among the five evaluated models. The improvements from Linear to MLP and then to CNN are consistent with the usefulness of nonlinear representations and convolutional spatial inductive bias. However, differences in model capacity and regularization prevent assigning the gains solely to those mechanisms.

The GRU reaches 90.26% mean test macro-F1 with 61,962 parameters, compared with 89.37% and 71,242 parameters for the row-token Transformer. This favors the tested GRU configuration, not recurrence over self-attention in general. Across the models, upper-body garments—especially Shirt—remain difficult to distinguish, indicating that strong aggregate scores do not imply uniform class-wise performance.

In the seed-42 representation ablation, rows yield the highest best validation accuracy for both GRU (91.19%) and Transformer (90.63%), ahead of columns and 4×4 patches. Future work should repeat the six representation configurations across multiple seeds and compare additional aggregation choices, such as a classification token, under matched experimental settings. Such tests would clarify which improvements arise from input representation rather than differences in model size or optimization.

4 Reproducibility and Project Artifacts

The experiments use a common stratified data split, fixed evaluation metrics, three documented training seeds, and validation-based checkpoint selection. Model configurations and instructions to reproduce the results are provided with the accompanying source code. The same split manifest is reused across all five models.

The implementation records Python 3.11.15, PyTorch 2.5.1+cu121, torchvision 0.20.1+cu121, and an NVIDIA GeForce RTX 4050 Laptop GPU. The dependency file also records NumPy 2.2.6, pandas 2.3.3, scikit-learn 1.7.2, Matplotlib 3.10.8, Seaborn 0.13.2, PyYAML 6.0.3, and tqdm 4.67.3. A fresh reproduction should use the same split manifest, model YAML file, training seed, device setting, and checkpoint path, then evaluate the saved `best.pt` checkpoint with the test-evaluation script. The commands below are written for a Python shell from the `assignment1/` directory; they do not depend on a PowerShell loop.

```
python src/eda.py --config configs/models/cnn.yaml --output-dir results/eda

python src/train.py --config configs/models/linear.yaml --seed 42 --device cuda --num-workers 0
↪  --output-dir results_norm_full --run-name linear_norm_seed42
python src/evaluate.py --config configs/models/linear.yaml --checkpoint
↪  checkpoints/linear/linear_norm_seed42/best.pt --device cuda --num-workers 0 --output-dir
↪  results_norm_full --allow-test
```

The train/evaluate pair is repeated for `linear`, `mlp`, `cnn`, `rnn`, and `transformer`, with seeds `42`, `123`, and `2026`. The config path and checkpoint model directory must match the model name. After all runs have completed, the aggregate comparison and training-curve figure are regenerated with:

```
python src/summarize_results.py --results-dir results_norm_full --output-file
↪  results/model_comparison.csv --curves-output results/combined_training_curves.png
```

- Repository: `https://github.com/tapstoptap/deep-learning-261`
- Environment: Python 3.11.15, PyTorch 2.5.1+cu121, NVIDIA RTX 4050 Laptop GPU
- Configuration: `configs/models/<model>.yaml`, split seed 42, training seeds 42/123/2026
- Results: normalized artifacts in `results_norm_full/` and EDA in `results/eda/`
- Reproduction instructions: `assignment1/README.md`

5 Contributions and AI Usage Disclosure

5.1 Member Contributions

The current contribution allocation for Assignment 1 is recorded as follows. Nguyen Minh Quang is the sole author and contributor to Assignment 1; Nguyen Minh Phuc and Nguyen Thanh The did not contribute to this assignment.

Table 11: Member contributions to Assignment 1.

Member	Student ID	Contribution
Nguyen Minh Quang	2453052	Complete responsibility for Assignment 1, including EDA, dataset and DataLoader preparation, preprocessing, implementation of the Linear, MLP, CNN, GRU, and Transformer models, experiment execution, result aggregation, comparison and error analysis, report writing, slide preparation, and video presentation.

5.2 AI Usage Disclosure

OpenAI tools were used at several stages of Assignment 1 for planning, implementation support, result checking, analysis, and report organization. The table below records the relevant uses. AI assistance was used as a support for the work: the group reviewed the suggestions and checked the resulting code, analysis, and reported values against the course handbook, repository artifacts, tests, and saved experiment outputs.

Table 12: AI assistance used during Assignment 1.

No.	Member / stage	Purpose and affected sections	Assistance and verification
1	Nguyen Thanh The Repository initialization 8–9 Sep 2026	Reviewed repository structure and handbook compliance; affected `README.md`, documentation, and `AI_USAGE.md`.	Suggested documentation, Pages, disclosure, and reproducibility improvements. The suggestions, links, and Pages result were reviewed and tested by the student.
2	Nguyen Minh Quang Assignment 1 planning 9 Sep 2026	Planned the Assignment 1 configuration structure; affected model configuration files and protocol documentation.	Helped separate shared rules from model-specific settings and drafted YAML configurations and protocol notes. The structure was checked against the handbook and repository requirements.
3	Nguyen Minh Quang Assignment 1 implementation 19–20 Sep 2026	Supported the experiment workflow; affected `assignment1/src/`, `configs/`, and `tests/`.	Helped with CLI design, module responsibilities, configuration flow, checkpoint and result organization, reproducibility safeguards, and test cases. The implementation was reviewed locally, unit tests passed, and smoke runs were checked.
4	Nguyen Minh Quang Milestone experiments and report 20 Sep 2026	Checked experiment logs and organized benchmark artifacts; affected saved results, documentation, and milestone report sections.	Helped aggregate the three-seed results, check seed/checkpoint mappings, produce error-analysis statistics, and organize artifacts. The summaries were checked against histories, JSON files, confusion matrices, and the split manifest.
5	Nguyen Minh Quang Result analysis October 2026	Supported comparison of the five models and the ablation results; affected the results tables, training-curve summary, ablation discussion, and related report text.	Helped summarize and analyze the saved test metrics, histories, and figures. Reported values and interpretations were checked carefully against the saved summaries, histories, JSON files, confusion matrices, and aggregate CSV files.

The complete AI-use log is maintained in `AI_USAGE.md`. The final responsibility for the code, experiments, interpretation, and submitted report remains with the group members.

References

[1] H. Xiao, K. Rasul, and R. Vollgraf, *Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms*, arXiv:1708.07747, 2017. Dataset and license:

`https://github.com/zalandoresearch/fashion-mnist`.

[2] PyTorch Contributors, *PyTorch: An Open Source Machine Learning Framework*, 2019. `https://pytorch.org/`.

[3] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, 12, 2825–2830, 2011. `https://scikit-learn.org/`.

[4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*, Journal of Machine Learning Research, 15, 1929–1958, 2014.

[5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 86(11), 2278–2324, 1998.

[6] K. Cho et al., *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*, Proceedings of EMNLP, 1724–1734, 2014.

[7] A. Vaswani et al., *Attention Is All You Need*, Advances in Neural Information Processing Systems 30, 2017.

[8] A. Dosovitskiy et al., *An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale*, International Conference on Learning Representations, 2021.

[9] D. P. Kingma and J. Ba, *Adam: A Method for Stochastic Optimization*, International Conference on Learning Representations, 2015.

[10] L. Prechelt, *Early Stopping—But When?*, In *Neural Networks: Tricks of the Trade*, Springer, 1998.

[11] M. Sokolova and G. Lapalme, *A Systematic Analysis of Performance Measures for Classification Tasks*, Information Processing & Management, 45(4), 427–437, 2009.